



BREACHWATCH

A data breach monitor for email addresses and company domains.
Project documentation and presentation guide.

Live app <https://breachwatch-production.up.railway.app>

Demo login analyst@breachwatch.dev / Demo1234!

Stack [Next.js 16](#), [TypeScript](#), [Drizzle ORM](#), [PostgreSQL](#), [Railway](#)

Screenshots taken with `BREACHWATCH_TODAY=2026-10-04`

Contents

1. Overview
2. How the core logic works
3. Architecture and data model
4. Screen walkthrough
5. Mobile view
6. Running locally
7. Testing
8. Deployment
9. Five-minute presentation script

1. Overview

BreachWatch tells a person or a small company when their email addresses show up in a known data breach, how serious each exposure is, and what to do about it before a deadline. It looks and works like a small security operations console.

The problem

Billions of account records have leaked from breached services. Leaked passwords are tried on other sites (credential stuffing), and leaked names, phone numbers and dates of birth make phishing and identity fraud easier. Most people never find out which of their accounts were affected, and when they do, nothing tracks whether they acted on it.

What the app does

- **Monitors assets.** A user adds email addresses, or a whole domain to cover every address at a company.
- **Scans a leak index.** Each scan matches the assets against an index of leaked addresses (stored only as SHA-256 hashes) and records new findings, called exposures.
- **Scores and schedules.** Each exposure gets a severity from the data types that leaked and a remediation deadline from that severity. The dashboard turns open and overdue findings into a 0 to 100 risk score.
- **Guides the response.** Each finding has a checklist built from the leaked data types (change password, enable MFA, contact the bank, and so on), notes, and a resolve button.
- **Breach intel.** A searchable catalog of 885 real, verified breaches from the public Have I Been Pwned breach list.
- **Password check.** A live check of any password against the Pwned Passwords corpus using k-anonymity, so the password never leaves the server.

Real data and demo data

The breach catalog and the password check use **real** public data. The per-email leak index is **demo data**: the real per-address lookup (HIBP breachedaccount API) needs a paid key. The app supports that API through the optional `HIBP_API_KEY` variable; without it, scans use the seeded leak index. All demo addresses use reserved `.example` domains.

2. How the core logic works

All business rules are pure functions in `src/lib/`. They take "today" as an argument and never read the clock themselves, so demos and tests are deterministic. `getToday()` returns `BREACHWATCH_TODAY` when it is set (format YYYY-MM-DD) and the real UTC date otherwise.

Severity (`src/lib/severity.ts`)

A breach takes the severity of the most dangerous data type it exposed. The same severity is copied onto every exposure from that breach.

Severity	Triggered by (examples)	Deadline	Risk points
CRITICAL	Passwords, auth tokens, credit cards, bank accounts, government IDs, security answers	3 days	12
HIGH	Dates of birth, phone numbers, addresses, partial card data, health data, private messages	7 days	6
MEDIUM	Names, usernames, IP addresses, locations, employers	14 days	2
LOW	Only the email address or other low-risk data	30 days	1

Status (`src/lib/exposure.ts`)

```
dueOn = detectedOn + deadline days for the severity
```

```
status(exposure, today):  
  resolvedOn is set      → RESOLVED  
  today is after dueOn   → OVERDUE  
  dueOn is 0 to 2 days away → DUE SOON  
  otherwise              → OPEN
```

A finding resolved after its deadline still counts as resolved, but the detail page labels it "resolved late".

Risk score and threat level

```
score = sum over unresolved exposures of  
        risk points for its severity x 1.5 if overdue  
score = min(100, round(score))  
  
threat level: 0-14 LOW, 15-44 GUARDED, 45-74 ELEVATED, 75-100 SEVERE
```

Example from the demo data: 3 critical (2 overdue), 2 high (1 overdue) and 2 medium gives $18 + 18 + 12 + 9 + 6 + 2 + 2 = 67$, which is ELEVATED. After the demo scan adds four findings the score passes 75 and the level becomes SEVERE.

Remediation queue order

Overdue first, then due soon, then open, then resolved. Inside each group: critical before high before medium before low, then earliest deadline first.

Scanning (src/server/scan.ts)

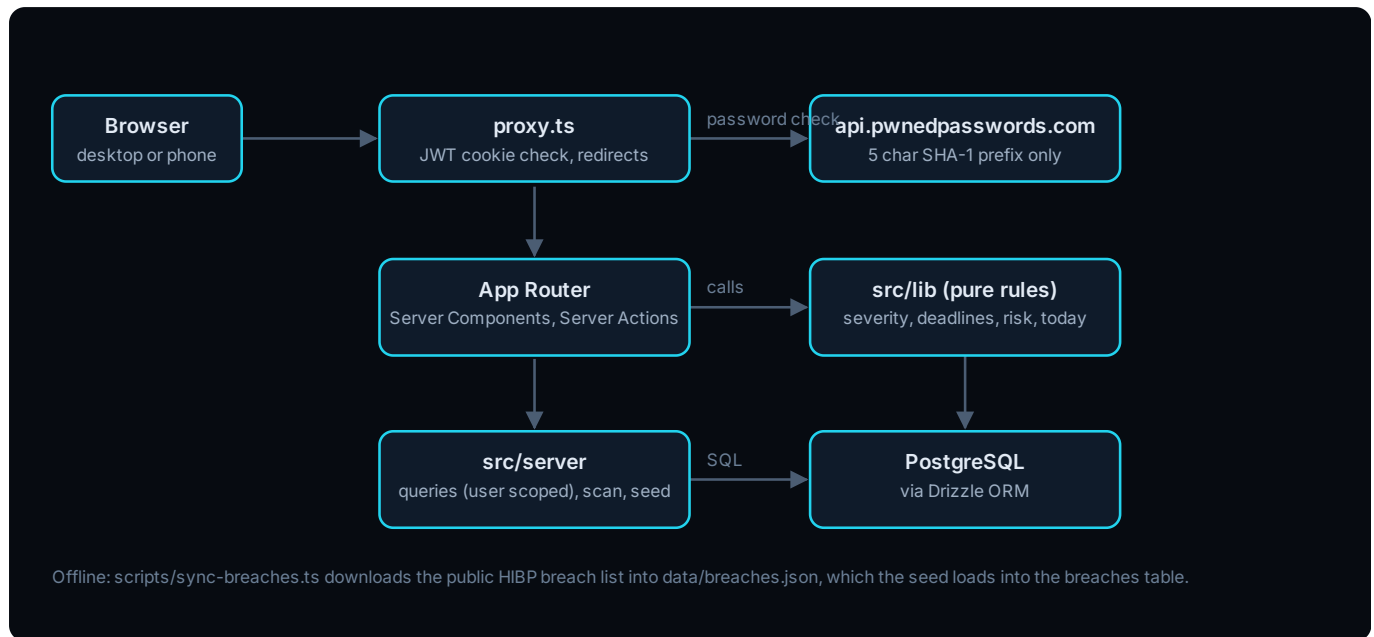
1. Load the user's assets. Email assets go before domain assets.
2. Email asset: hash the normalized address with SHA-256 and look it up in `leak_records.email_hash`. If `HIBP_API_KEY` is set, the live HIBP API is also asked and its breach names are matched to the catalog.
3. Domain asset: find every leak record whose `email_domain` equals the domain.
4. Insert one exposure per (user, breach, address hash) with `ON CONFLICT DO NOTHING`, so repeated scans never create duplicates and an address monitored directly owns its findings.
5. Write a row to `scans` and return a log that the dashboard console prints line by line.

Password check (src/lib/password.ts)

```
sha1("password123") = CBFDAC6008F9CAB4083784CBD1874F76618D2A97
send:      GET https://api.pwnedpasswords.com/range/CBFDA  (prefix, 5 chars)
receive:   ~800 lines of "SUFFIX:COUNT" (padded)
compare:   find C6008F9CAB4083784CBD1874F76618D2A97 locally → count
```

Any count above zero means "compromised, do not use". If the API cannot be reached, only the strength estimate is shown and the page says so.

3. Architecture and data model



Request flow

- **proxy.ts** (Next.js 16 replaces middleware with proxy) reads the session cookie and redirects signed-out visitors to /sign-in, and signed-in users away from it. It does not touch the database.
- **Pages** are async Server Components. Each one calls `requireUser()`, which verifies the JWT and loads the user, then reads data through `src/server/queries.ts`.
- **Mutations** are Server Actions in `src/app/actions/`. Forms validate with Zod and return field errors that render inline. Selects are uncontrolled (`defaultValue`) because React 19 resets forms after an action.
- **Every query that touches user data takes `userId` and filters on it.** Integration tests check that a second user cannot read, change or delete the demo user's data.

Authentication

Passwords are hashed with `bcryptjs` (cost 10). On sign in the server signs an HS256 JWT with `jose` (subject = user id, 8 hour expiry) and stores it in an HTTP-only, SameSite=Lax cookie, marked Secure in production. Unknown email and wrong password give the same message. Responses also carry security headers: X-Frame-Options DENY, nosniff, HSTS, Referrer-Policy and Permissions-Policy.

Tables (src/db/schema.ts, migration drizzle/0000_init.sql)

Table	Purpose	Key columns
users	Accounts	email (unique), password_hash
breaches	Public breach catalog, shared	name (unique), breach_date, pwn_count, data_classes text[], severity
leak_records	Threat intel index, shared	email_hash (SHA-256), email_masked, email_domain, breach_id
assets	What a user monitors	user_id, kind (email or domain), value, label, last_scanned_on; unique (user_id, kind, value)
exposures	Findings	user_id, asset_id, breach_id, severity, detected_on, due_on, resolved_on, steps_done text[], notes; unique (user_id, breach_id, email_hash)
scans	Scan audit log	user_id, ran_on, assets_checked, records_matched, new_exposures

Deleting a user cascades to assets, exposures and scans. Deleting an asset cascades to its exposures. The leak index stores no plain email addresses, only a hash and a masked form such as `ad*****@example.com`.

Project layout

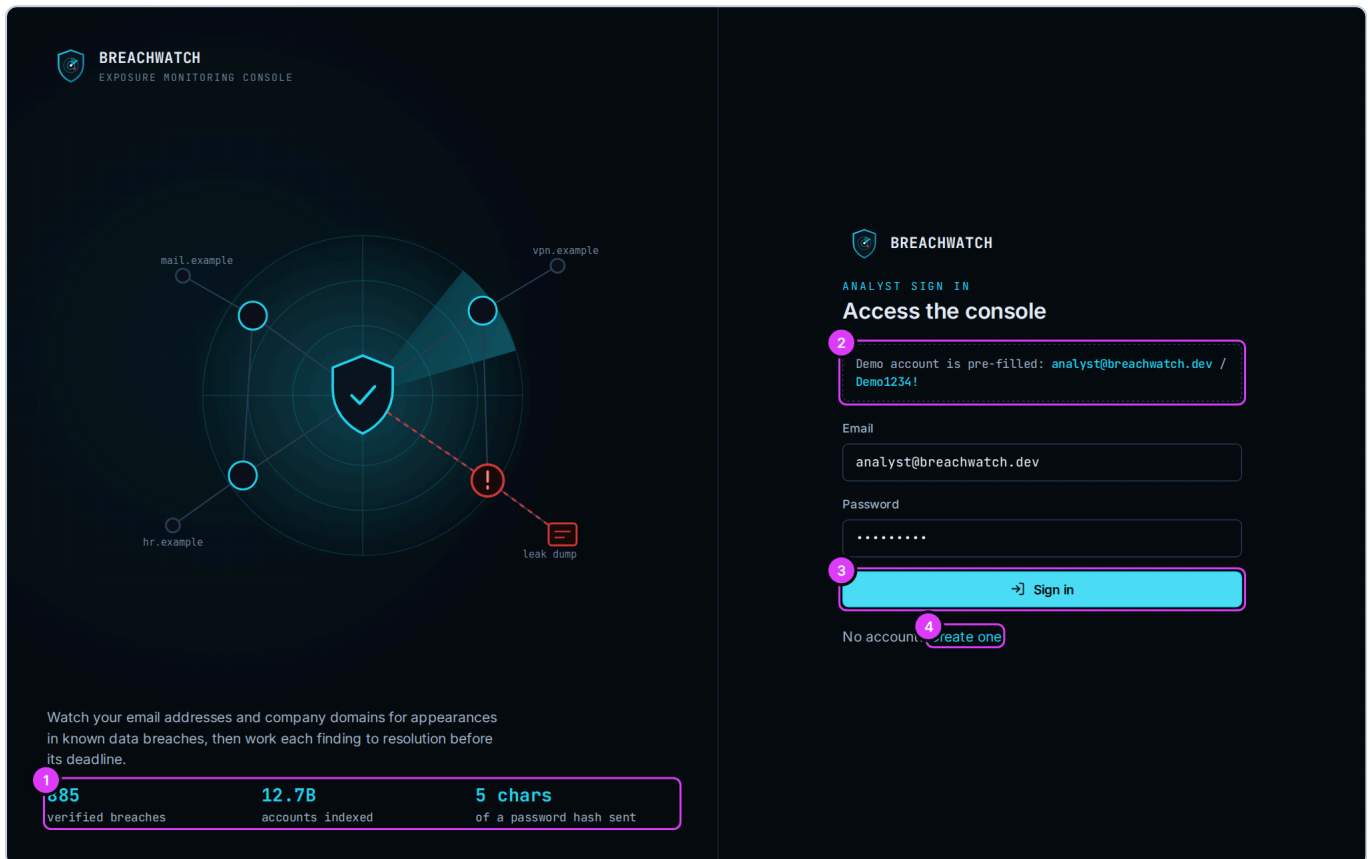
src/lib/	Pure rules: dates, today, severity, exposure status and risk, remediation steps, password, identity, validation, auth tokens
src/server/	Database work: queries (user scoped), scan, catalog loader, seed, session
src/app/	Routes: (auth) sign-in and sign-up, (app) dashboard, assets, exposures, breaches, password-check, api/health
src/components/	Badges, threat gauge, severity bars, scan console, form helpers
scripts/	migrate, seed, sync-breaches
tests/	unit, integration (real Postgres) and e2e (Playwright)

4. Screen walkthrough

Numbered pink markers on each screenshot match the numbered notes under it.

Sign in

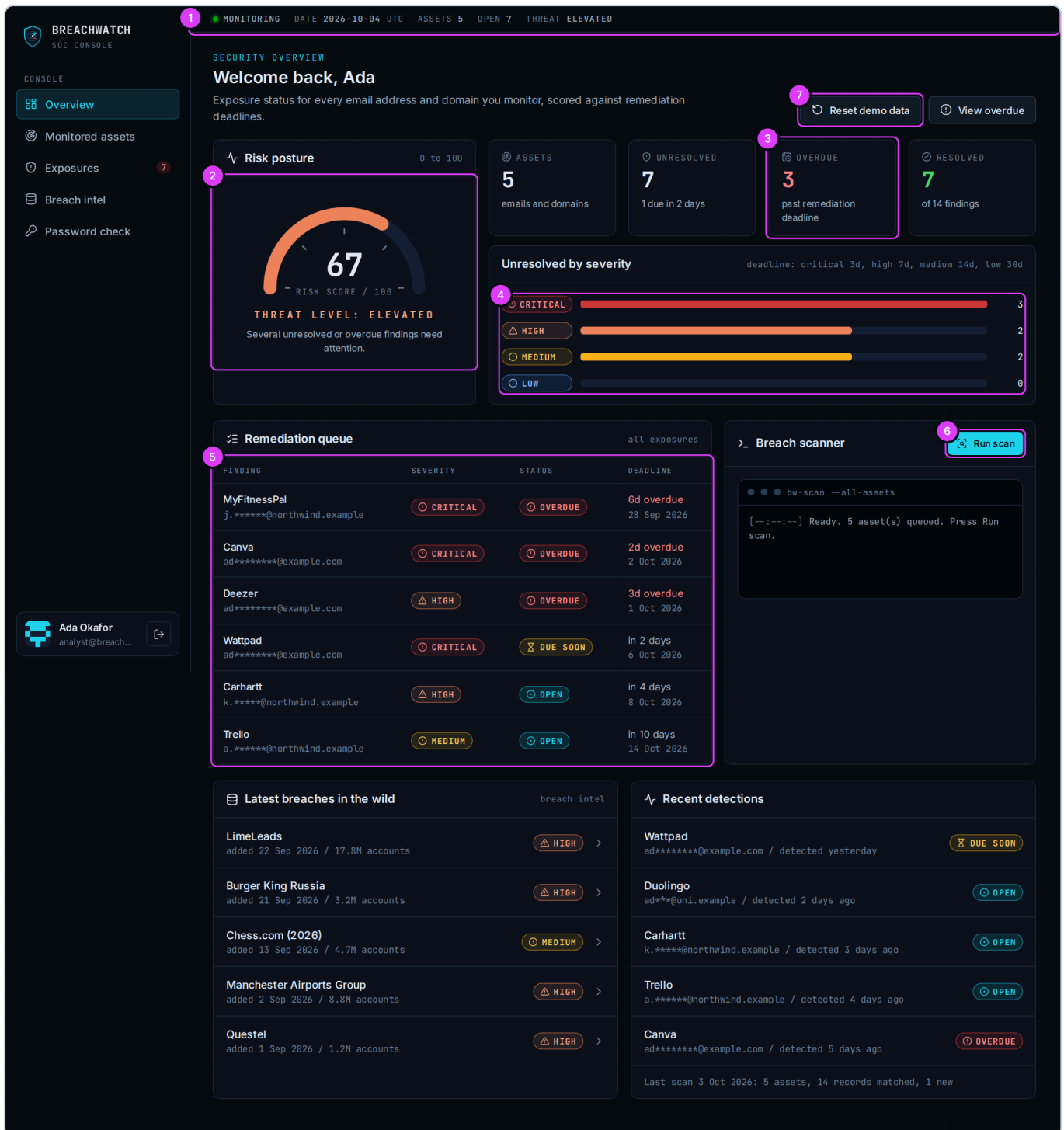
The entry point. Every other page redirects here until a valid session cookie is present.



- 1 Live catalog totals read from the database: verified breaches and accounts covered.
- 2 The demo credentials are shown and already filled in, so the presenter only presses Sign in.
- 3 Submits a Server Action. Input is validated with Zod and errors appear inline under each field. A wrong password shows one generic message so accounts cannot be enumerated.
- 4 New users can create an account. A new account starts with no assets and cannot see anyone else's data.

Overview dashboard

The landing page after sign in. It answers one question: how exposed am I right now, and what should I fix first?



- 1 Status bar: the business date in use (BREACHWATCH_TODAY), asset count, unresolved findings and the current threat level.
- 2 Risk score from 0 to 100 and the threat level it maps to (low, guarded, elevated, severe).
- 3 Stat tiles: assets, unresolved, overdue (past the remediation deadline) and resolved findings.
- 4 Unresolved findings by severity. Every bar carries its label and count, so color is never the only signal.
- 5 Remediation queue: overdue first, then by severity, then by deadline. Each row links to the finding.

- 6 Run scan checks every monitored asset against the leak index and records new findings.
- 7 Reset demo data (demo account only) rebuilds the account's assets and findings relative to today, so every presentation starts from the same state.

Running a scan

The demo data contains leak records that earlier scans have not seen yet, so the first scan in a demo always finds something.

BREACHWATCH SOC CONSOLE

CONSOLE

- Overview
- Monitored assets
- Exposures 11
- Breach intel
- Password check

Welcome back, Ada

Exposure status for every email address and domain you monitor, scored against remediation deadlines.

Risk posture 0 to 100

93 RISK SCORE / 100

THREAT LEVEL: SEVERE

Critical exposures are open past their deadline. Act today.

ASSETS 5 emails and domains

UNRESOLVED 11 1 due in 2 days

OVERDUE 3 past remediation deadline

RESOLVED 7 of 18 findings

Unresolved by severity deadline: critical 3d, high 7d, medium 14d, low 30d

- CRITICAL 4
- HIGH 4
- MEDIUM 3
- LOW 0

Remediation queue all exposures

FINDING	SEVERITY	STATUS	DEADLINE
MyFitnessPal j.*****@northwind.example	CRITICAL	OVERDUE	6d overdue 28 Sep 2026
Canva ad*****@example.com	CRITICAL	OVERDUE	2d overdue 2 Oct 2026
Deezer ad*****@example.com	HIGH	OVERDUE	3d overdue 1 Oct 2026
Wattpad ad*****@example.com	CRITICAL	DUE SOON	in 2 days 6 Oct 2026
Fanlore ad*****@example.com	CRITICAL	OPEN	in 3 days 7 Oct 2026

Breach scanner Run scan

```

1  [19:36:14] Connecting to leak index and breach catalog...
   [19:36:15] Loaded 5 monitored asset(s)
   [19:36:15] NEW CRITICAL ad*****@example.com found in Fanlore
   [19:36:15] ad*****@example.com: 7 record(s) in leak index, 1 new
   [19:36:15] NEW HIGH a.*****@northwind.example found in RingCentral
   [19:36:15] a.*****@northwind.example: 4 record(s) in leak index, 1 new
   [19:36:15] NEW MEDIUM ad**@uni.example found in Chess2026
   [19:36:15] ad**@uni.example: 3 record(s) in
  
```

Ada Okafor
analyst@breach... [→]

- 1 Scan log. Red lines are new findings with their severity, masked address and breach. The summary line counts new exposures.
- 2 Scanning is idempotent: running it again adds nothing until the leak index changes.

Monitored assets

The user's attack surface: email addresses and whole domains.

BREACHWATCH
SOC CONSOLE

CONSOLE

- Overview
- Monitored assets**
- Exposures 11
- Breach intel
- Password check

ATTACK SURFACE
Monitored assets

Email addresses are matched by SHA-256 hash against the leak index. A domain matches every leaked address at that domain, so you can watch a whole team. Up to 25 assets.

Add an asset a scan runs as soon as it is added

1 /pe Email address or domain Label (optional)
Email name@example.com or example.com Work, Personal... **+ Add and scan**

5 asset(s)

ASSET	FINDINGS	WORST STATUS	LAST SCANNED
ada.okafor@example.com email / Personal	4 open / 7	OVERDUE	4 Oct 2026
a.okafor@northwind.example email / Work	2 open / 4	OPEN	4 Oct 2026
ada.o@uni.example email / Student	2 open / 3	OPEN	4 Oct 2026
ada.backup@example.org email / Recovery	0 open / 0	CLEAN	4 Oct 2026
*@northwind.example domain / Company domain	3 open / 4	OVERDUE	4 Oct 2026

Ada Okafor
analyst@breach...

- 1 Add an email or a domain. Values are normalized (lowercased, trimmed) and validated with Zod. A scan runs immediately after adding.
- 2 A domain asset (*@northwind.example) matches every leaked address at that domain, which is how a company watches its staff.
- 3 Worst status per asset. The recovery address is clean: it appears in no breach.
- 4 Stop monitoring. Asks for confirmation, then deletes the asset and its findings (scoped to the signed-in user).

Exposures

Every finding, one row per identity per breach. Filters live in the URL, so a filtered view can be bookmarked.

BREACHWATCH
SOC CONSOLE

MONITORING DATE 2026-10-04 UTC ASSETS 5 OPEN 11 THREAT SEVERE

FINDINGS
Exposures

Each row is one monitored identity found in one breach. Overdue findings are listed first, then by severity and deadline.

1 Filter by status, severity or free text. This view shows all unresolved findings.

2 of 18 exposure(s)

3 Status

4 Deadline

BREACH	SEVERITY	STATUS	ASSET	DETECTED	DEADLINE
MyFitnessPal j.*****@northwind.example	CRITICAL	OVERDUE	Company domain *@northwind.example	25 Sep 2026	6d overdue 28 Sep 2026
Canva ad*****@example.com	CRITICAL	OVERDUE	Personal ada.okafor@example.com	29 Sep 2026	2d overdue 2 Oct 2026
Deezer ad*****@example.com	HIGH	OVERDUE	Personal ada.okafor@example.com	24 Sep 2026	3d overdue 1 Oct 2026
Wattpad ad*****@example.com	CRITICAL	DUE SOON	Personal ada.okafor@example.com	3 Oct 2026	in 2 days 6 Oct 2026
Fanlore ad*****@example.com	CRITICAL	OPEN	Personal ada.okafor@example.com	4 Oct 2026	in 3 days 7 Oct 2026
Carhartt k.*****@northwind.example	HIGH	OPEN	Company domain *@northwind.example	1 Oct 2026	in 4 days 8 Oct 2026
Manchester Airports Group t.*****@northwind.example	HIGH	OPEN	Company domain *@northwind.example	4 Oct 2026	in 7 days 11 Oct 2026
RingCentral a.*****@northwind.example	HIGH	OPEN	Work a.okafor@northwind.example	4 Oct 2026	in 7 days 11 Oct 2026
Trello a.*****@northwind.example	MEDIUM	OPEN	Work a.okafor@northwind.example	30 Sep 2026	in 10 days 14 Oct 2026
Duolingo ad**@uni.example	MEDIUM	OPEN	Student ada.o@uni.example	2 Oct 2026	in 12 days 16 Oct 2026
Chess.com (2026) ad**@uni.example	MEDIUM	OPEN	Student ada.o@uni.example	4 Oct 2026	in 14 days 18 Oct 2026

Ada Okafor
analyst@breach...

- 1 Filter by status, severity or free text. This view shows all unresolved findings.
- 2 How many findings match out of the total.
- 3 Status computed from today's date: overdue, due soon (2 days or less), open or resolved.
- 4 Deadline column: days overdue in red, otherwise the time left and the due date.

Finding detail

Where the analyst works a finding to resolution.

BREACHWATCH
SOC CONSOLE

MONITORING DATE 2026-10-04 UTC ASSETS 5 OPEN 11 THREAT SEVERE

< All exposures

FINDING #4
ad***@example.com in Canva**

1 **CRITICAL** **OVERDUE** via ada.okafor@example.com (Personal) 4 **Mark resolved**

Remediation deadline critical findings must be resolved within 3 days

2

DETECTED 29 Sep 2026	DEADLINE 2 Oct 2026 2 DAYS OVERDUE	TODAY 4 Oct 2026
-------------------------	--	---------------------

Response checklist 1/4 done

3

- ☒ **Change the password on the breached service**
Use a new, unique password generated by a password manager.
- ☐ **Replace the password everywhere it was reused**
Attackers try leaked passwords on other sites (credential stuffing).
- ☐ **Turn on multi-factor authentication**
An authenticator app or security key stops logins with a stolen password.
- ☐ **Expect targeted phishing**
Leaked names, phone numbers and addresses make scam messages look real. Verify before you click.

Analyst notes

Changed Canva password. Still need to check other sites that used the same one.

Save notes

Breach intel catalog entry

SERVICE canva.com
BREACH DATE 24 May 2019
ACCOUNTS 137.3M

5 **DATA EXPOSED**

- Email addresses
- Geographic locations
- Names
- Passwords
- Usernames

In May 2019, the graphic design tool website Canva suffered a data breach that impacted 137 million subscribers. The exposed data included email addresses, usernames, names, cities of residence and passwords stored as bcrypt hashes for users not using social logins.

- 1 Severity and status badges, plus the asset that produced this finding.
- 2 Deadline timeline: detected date, deadline (detected + SLA days) and today. Overdue deadlines are outlined in red.
- 3 Response checklist generated from the data types exposed. Passwords lead to password change, reuse check and MFA steps.
- 4 Mark resolved stamps today's date. Resolved after the deadline is reported as resolved late.
- 5 Data exposed in the breach, each tagged with the severity that data type carries.

Breach intel catalog

A searchable copy of the public Have I Been Pwned breach list, newest breach first.

BREACHWATCH

SOC CONSOLE

CONSOLE

Overview

Monitored assets

Exposures11

Breach intel

Password check

MONITORING

DATE 2026-10-04 UTC

ASSETS 5

OPEN 11

THREAT SEVERE

THREAT INTELLIGENCE

Breach catalog

885 verified public breaches covering 12.7B accounts, from the Have I Been Pwned breach list (CC BY 4.0). Severity is set by the most sensitive data class exposed.

Search service or domain

Severity

ALL

Search

885 result(s), newest breach first

27 Aug 2026

8.8M accts

Manchester Airports Group

magairports.com

HIGH

8 data types

15 Aug 2026

2.0M accts

Oz Hair and Beauty

ozhairandbeauty.com

HIGH

5 data types

13 Aug 2026

12.9M accts

Carhartt

carhartt.com

HIGH

4 data types

6 Aug 2026

144.5K accts

Fanlore

fanlore.org

CRITICAL

4 data types

3 Aug 2026

4.7M accts

Chess.com (2026)

chess.com

MEDIUM

4 data types

1 Aug 2026

1.2M accts

Questel

questel.com

HIGH

7 data types

1 Aug 2026

218.4K accts

Alcon

alcon.com

HIGH

4 data types

27 Jul 2026

1.6M accts

RingCentral

ringcentral.com

HIGH

4 data types

21 Jul 2026

865.3K accts

SplitVPN

splitvpn.io

HIGH

5 data types

13 Jul 2026

732.2K accts

Brinks Home

brinkshome.com

1 Jul 2026

821.1K accts

Fluke

fluke.com

18 Jun 2026

276.1K accts

Inter-Con Security

icsecurity.com

Ada Okafor

analyst@breach...

- 1 Catalog size and source. The list is refreshed with npm run breaches:sync.
- 2 Search by service name or domain and filter by severity.
- 3 Breach card: date, accounts affected, domain and severity derived from the exposed data types.

Breach detail

Everything known about one breach, and whether it affects the signed-in user.

The screenshot displays the BreachWatch SOC Console interface. The left sidebar contains navigation links: Overview, Monitored assets, Exposures (with a red badge showing 11), Breach intel (highlighted), and Password check. The main content area shows the 'Breach catalog' for 'LINKEDIN.COM'. At the top, a status bar indicates 'MONITORING', 'DATE 2026-10-04 UTC', 'ASSETS 5', 'OPEN 11', and 'THREAT SEVERE'. The breach is labeled 'CRITICAL' with a 'deadline when detected: 3 days'. A summary section (1) contains four key facts: 'ACCOUNTS 164,611,595', 'BREACH DATE 5 May 2012', 'MADE PUBLIC 21 May 2016', and 'DATA TYPES 2'. Below this, the 'What happened' section (2) describes the incident: 'In May 2016, LinkedIn had 164 million email addresses and passwords exposed. Originally hacked in 2012, the data remained out of sight until being offered for sale on a dark market site 4 years later. The passwords in the breach were stored as SHA1 hashes without salt, the vast majority of which were quickly cracked in the days following the release of the data.' It includes tabs for 'Email addresses' and 'Passwords', and a 'RECOMMENDED RESPONSE' list: 'Change the password on the breached service', 'Replace the password everywhere it was reused', 'Turn on multi-factor authentication', and 'Expect targeted phishing'. The 'Your exposure' section (3) shows '1 of your identities are in this breach' with an example email 'ad*****@example.com' and a 'RESOLVED' status.

- 1 Key facts: accounts, breach date, date made public and number of data types.
- 2 Description of the incident from the catalog.
- 3 Your exposure: the user's own findings in this breach, with status. Other users' data is never shown.

Password check

A live check against Pwned Passwords. This is real data, not seed data.

The screenshot displays the BreachWatch SOC Console interface. On the left is a sidebar with navigation links: Overview, Monitored assets, Exposures (with a red badge showing 11), Breach intel, and Password check (highlighted). The main content area is titled 'Has this password been leaked?' under the 'CREDENTIAL HYGIENE' section. It includes a 'Check a password' form with a text input field labeled 'Type a password to test', a 'Check password' button, and a hint: 'Try a common one like password123 to see a hit.' To the right of the form is a 'Result' panel. It shows a large red number '2,266,543' indicating the number of times the password has been seen in data breaches, with a warning 'Do not use this password.' Below this, it states 'Strength: Very weak' with a progress bar and 'about 10 bits'. It also provides a hint: 'Use at least 12 characters. Add uppercase letters. Add symbols.' At the bottom of the result panel, it shows the SHA-1 hash 'CBFDA*****8D2A97' and explains the k-anonymity process: 'Sent: GET /range/CBFDA. The rest of the hash was compared on the server.'

1 password

Type a password to test

Check password

Try a common one like password123 to see a hit.

2 **2,266,543**
times seen in data breaches. Do not use this password.

3 Strength: Very weak about 10 bits
Use at least 12 characters. Add uppercase letters. Add symbols.

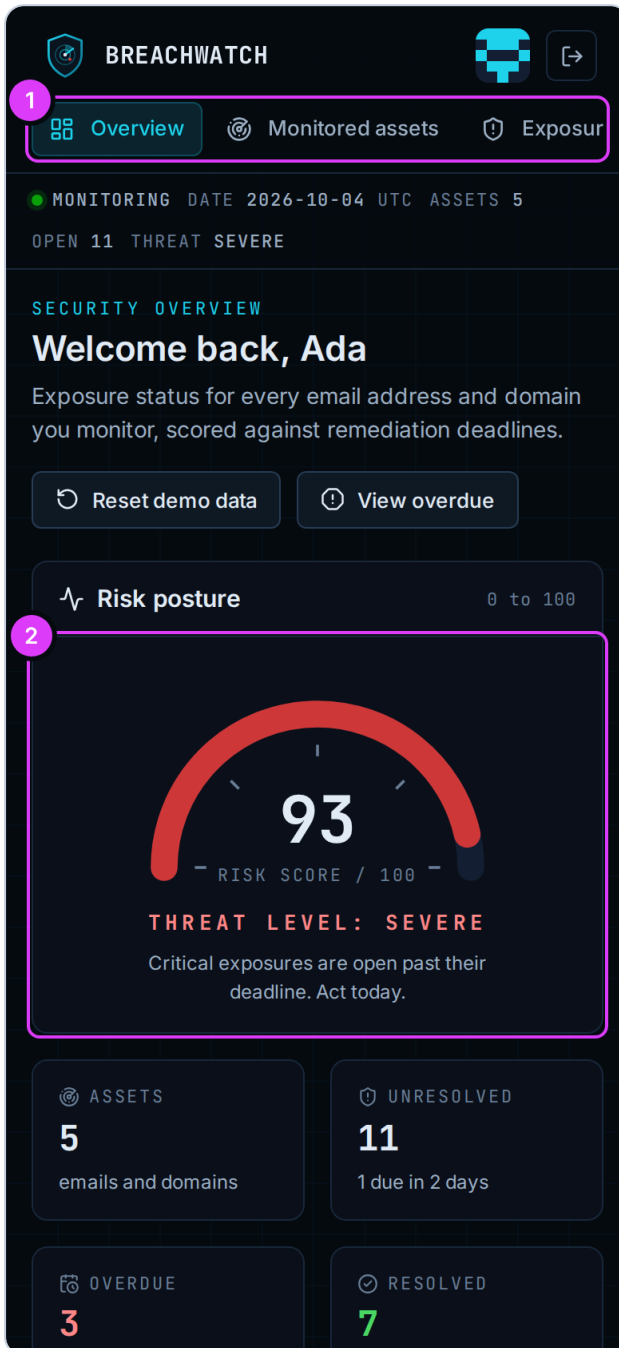
4 SHA-1 = CBFDA*****8D2A97
Sent: GET /range/CBFDA. The rest of the hash was compared on the server.

- 1** The password is posted to a Server Action. It is never stored or logged.
- 2** Verdict: the number of times this password appears in breach corpora. Any appearance means do not use it.
- 3** Local strength estimate (length and character variety), with hints.
- 4** k-anonymity in action: only the first 5 characters of the SHA-1 hash were sent to the API. The rest was compared on the server.

5. Mobile view

The layout is a single CSS grid that collapses below 860px. Grids use `minmax(0, 1fr)` so long values cannot push the page wider than the screen, and the e2e mobile test asserts that the page never scrolls sideways on a Pixel 7.

These two captures were taken after the demo scan, so the dashboard shows the SEVERE level.



Mobile: overview



Mobile: exposures

- 1 The sidebar becomes a top bar with a horizontally scrolling menu.
- 2 Panels stack into one column.
- 3 On phones, severity and status move under the breach name so the table fits without sideways scrolling.

6. Running locally

Requirements: Node.js 22 and PostgreSQL 14 or newer.

```
service postgresql start
sudo -u postgres createdb breachwatch
sudo -u postgres createdb breachwatch_test

cp .env.example .env           # set DATABASE_URL and SESSION_SECRET
npm install
npm run db:migrate             # apply drizzle/ migrations
npm run db:seed                # reset and load demo data
npm run dev                    # http://localhost:3000
```

Variable	Required	Meaning
DATABASE_URL	yes	Postgres connection string
SESSION_SECRET	in production	Signs session cookies (16+ characters)
BREACHWATCH_TODAY	no	Pins "today" (YYYY-MM-DD) for demos and tests
HIBP_API_KEY	no	Enables live per-email lookups during scans

Scripts

npm run dev / build / start	Next.js development server, production build, production server
npm run start:prod	Migrate, seed only if empty, then start (used by Railway)
npm run db:generate	Create a new SQL migration from schema changes (drizzle-kit)
npm run breaches:sync	Refresh data/breaches.json from the public HIBP breach list
npm test	Unit and integration tests (Vitest)
npm run test:e2e	Playwright tests (builds and starts the app on port 3100)
npm run docs:pdf	Rebuild this PDF (needs a prior npm run build)

7. Testing

Suite	Tool	Tests	What it covers
Unit	Vitest	47	Dates and today, severity, deadlines, status, risk score and levels, queue order, k-anonymity parsing, password strength, email hashing and masking, Zod schemas, remediation steps, text cleaning, JWT sign and verify (including expiry and wrong secret)
Integration	Vitest + real Postgres (breachwatch_test)	17	Seed produces every status, dashboard summary, catalog search, scan finds pending records, scan is idempotent, live lookup used and its failure tolerated, user scoping on every read and write, duplicate assets, resolve and reopen, cascade delete, demo reset restores the starting state and leaves other users alone
End to end, desktop	Playwright, Chromium 1440×900	14	Redirects, pre-filled demo login, wrong password, sign-up inline errors, new user isolation (404 on another user's finding), sign out, dashboard, scan, reset demo data, add asset with validation, delete asset, filters, checklist, notes, resolve, catalog, live password check
End to end, mobile	Playwright, Pixel 7	1	Sign in, dashboard, navigation, finding detail, no horizontal overflow
Total		79	All passing before deployment

The integration suite drops and recreates the test schema from the committed migrations before running, so it also proves the migrations apply cleanly. The e2e suite reseeds the database through the same seed code before each spec and pins `BREACHWATCH_TODAY=2026-10-04`.

```
npm test          # 64 unit + integration tests
npm run test:e2e  # 15 Playwright tests (desktop + mobile)
```

8. Deployment (Railway)

The app runs in the Railway project **school-projects** as the service **breachwatch**, with its own database service **breachwatch-postgres**. Railway builds from the GitHub repository with Railpack (Node 22 from `engines`).

```
{
  "build": { "builder": "RAILPACK", "buildCommand": "npm run build" },
  "deploy": { "startCommand": "npm run start:prod",
    "healthcheckPath": "/api/health", "healthcheckTimeout": 120,
    "restartPolicyType": "ON_FAILURE" }
}
```

- **Start command:** `tsx scripts/migrate.ts && tsx scripts/seed.ts --if-empty && next start -H 0.0.0.0`. Migrations always run; demo data is loaded only when the users table is empty, so redeploys never wipe data.
- **Variables:** `DATABASE_URL=${{breachwatch-postgres.DATABASE_URL}}` (a reference, resolved by Railway to the private network address), a random 64 character `SESSION_SECRET`, and `NODE_ENV=production`.
- **Health check:** `GET /api/health` runs `select 1` against the database and returns `{"status":"ok","database":"ok"}`, or HTTP 503 if the database is unreachable, so Railway only routes traffic to a healthy deploy.
- **Public URL:** `https://breachwatch-production.up.railway.app`

Redeploying

Push to the connected branch and Railway rebuilds automatically. Redeploys keep existing data. To put the demo account back to its starting state, sign in as the demo user and press **Reset demo data** on the dashboard. It only affects the demo account.

9. Five-minute presentation script

Before you start: sign in on the live URL and press **Reset demo data** on the dashboard. This rebuilds the demo account relative to today, so the numbers below match. Then sign out and leave the sign-in page open. Keep this PDF open as a backup.

Time	Do	Say
0:00 - 0:40	Sign-in page.	"Billions of accounts have leaked in data breaches, and most people never find out which of theirs were affected. BreachWatch monitors your email addresses and your company domain, tells you how serious each exposure is, and tracks the fix against a deadline. The catalog on the left is real: 885 verified breaches from Have I Been Pwned."
0:40 - 1:40	Press Sign in (the demo login is pre-filled). Point at the gauge, the tiles and the queue.	"This is the analyst console. The risk score is 67, ELEVATED, because three findings are past their deadline. Severity comes from what leaked: passwords are critical and must be fixed in 3 days, dates of birth are high with 7 days. The queue puts overdue critical findings first."
1:40 - 2:20	Press Run scan. Let the log print. Reload.	"A scan hashes each address and checks it against the leak index. It found four new exposures, including a password leak from Fanlore in August 2026, and the threat level is now SEVERE. Scanning twice adds nothing, it is idempotent."
2:20 - 3:10	Exposures, filter Overdue, open Canva. Tick Turn on MFA, then Mark resolved.	"Each finding has a deadline timeline and a checklist generated from the leaked data. Canva leaked passwords, so the steps are change it, replace reused passwords and turn on MFA. I mark it resolved; it was two days late, so the app records it as resolved late."
3:10 - 3:40	Monitored assets. Add demo.user@example.net.	"I can monitor a single address or a whole domain. Adding an asset triggers a scan straight away: this address is in four breaches. Input is validated, so a bad domain or a duplicate shows an inline error."
3:40 - 4:20	Password check. Type password123.	"This one is live. The password never leaves our server: we send only the first five characters of its SHA-1 hash, CBFDA, and compare the rest ourselves. That is k-anonymity. password123 has been seen over two million times."
4:20 - 5:00	Show the phone view (or this PDF, section 5) and finish on the dashboard.	"Built with Next.js 16 Server Components and Server Actions, Drizzle and Postgres, deployed on Railway with a health check that pings the database. Every query is scoped to the signed-in user. 79 automated tests cover the rules, the database and the screens on desktop and mobile. Thank you."

Likely questions

- **Is the email data real?** The breach catalog and password check are real. The per-email index is demo data because the real lookup API is paid; set HIBP_API_KEY to use it.
- **Do you store emails or passwords?** Passwords checked on the password page are never stored. The leak index stores only SHA-256 hashes and masked addresses. Account passwords are bcrypt hashes.
- **Why does the date say 4 Oct 2026?** Rules take "today" as a parameter, and BREACHWATCH_TODAY can pin it for repeatable demos and tests.